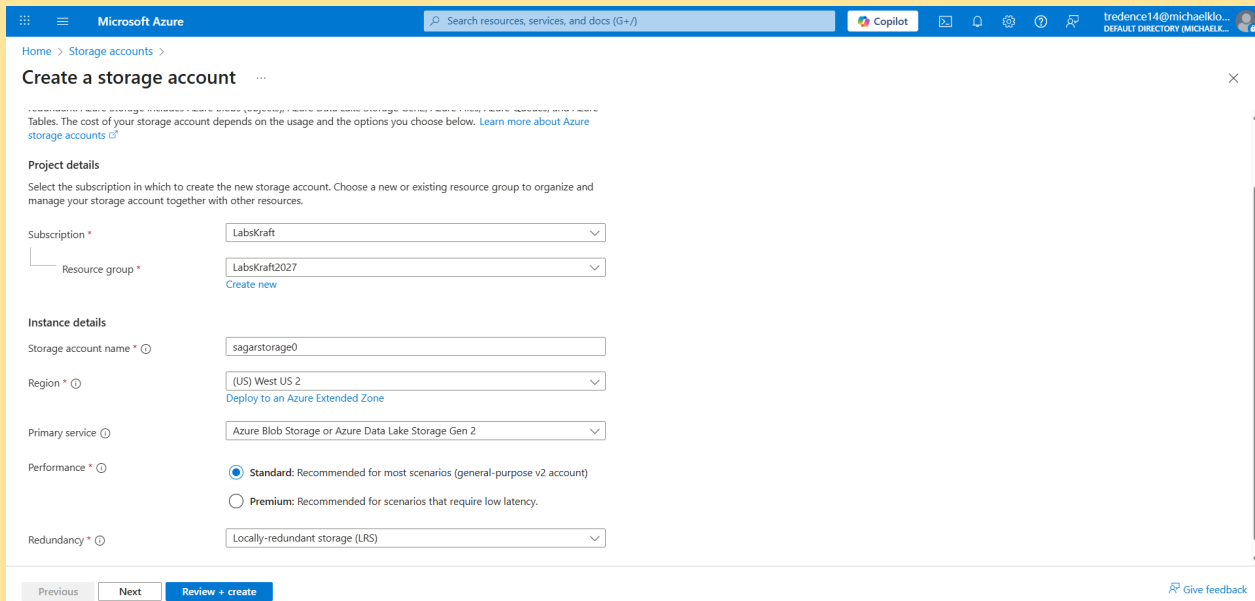


Project 02: Retail Compliance & Analytics Platform on Azure Databricks

Creation of Resources

adls storage for project:



The screenshot shows the 'Create a storage account' wizard in the Microsoft Azure portal. The breadcrumb trail indicates the user is in 'Home > Storage accounts'. The page title is 'Create a storage account'. A note states: 'Tables. The cost of your storage account depends on the usage and the options you choose below. [Learn more about Azure storage accounts](#)'. The form is divided into two sections: 'Project details' and 'Instance details'. In 'Project details', the 'Subscription' is 'LabsKraft' and the 'Resource group' is 'LabsKraft2027' (with a 'Create new' link). In 'Instance details', the 'Storage account name' is 'sagarstorage0', the 'Region' is '(US) West US 2' (with a 'Deploy to an Azure Extended Zone' link), the 'Primary service' is 'Azure Blob Storage or Azure Data Lake Storage Gen 2', the 'Performance' is 'Standard: Recommended for most scenarios (general-purpose v2 account)' (selected), and the 'Redundancy' is 'Locally-redundant storage (LRS)'. At the bottom, there are 'Previous', 'Next', and 'Review + create' buttons, along with a 'Give feedback' link.

Microsoft Azure

Search resources, services, and docs (G+/I)

Copilot

fredence14@michaelko...
DEFAULT DIRECTORY (MICHAELK...

Home > Storage accounts >

Create a storage account

Tables. The cost of your storage account depends on the usage and the options you choose below. [Learn more about Azure storage accounts](#)

Project details

Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.

Subscription * LabsKraft

Resource group * LabsKraft2027
[Create new](#)

Instance details

Storage account name * sagarstorage0

Region * (US) West US 2
[Deploy to an Azure Extended Zone](#)

Primary service Azure Blob Storage or Azure Data Lake Storage Gen 2

Performance * ☒ Standard: Recommended for most scenarios (general-purpose v2 account)
☐ Premium: Recommended for scenarios that require low latency.

Redundancy * Locally-redundant storage (LRS)

Previous Next Review + create

[Give feedback](#)

adls storage for sagar_cap2:

Home > Storage accounts >

Create a storage account

Review the cost of your storage account deployment on the page and the options you choose below. Learn more about Azure storage accounts.

Project details

Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.

Subscription * LabsKraft

Resource group * LabsKraft2027
[Create new](#)

Instance details

Storage account name * sagarcap2

Region * (US) West US 2
[Deploy to an Azure Extended Zone](#)

Primary service * Azure Blob Storage or Azure Data Lake Storage Gen 2

Performance * ☒ **Standard:** Recommended for most scenarios (general-purpose v2 account)
☐ **Premium:** Recommended for scenarios that require low latency.

Redundancy * Locally-redundant storage (LRS)

[Previous](#) [Next](#) [Review + create](#)

Access connector for catalog:

Microsoft Azure

Home > Access Connector for Azure Databricks >

Create an Access Connector for Azure Databricks

Basics Tags Managed Identity Review + create

The Azure Databricks Access Connector lets you connect managed identities to an Azure Databricks account for the purpose of accessing data registered in Unity Catalog.

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * LabsKraft

Resource group * LabsKraft2027
[Create new](#)

Instance details

Name * sagarconnector

Region * West US 2

[Previous](#) [Next](#) [Review + create](#)

[Give feedback](#)

Giving permission of storage blob contributor to Access connector:

Microsoft Azure

Home > Storage accounts > sagarcap2 | Access Control (IAM)

Add role assignment

Role **Members** Conditions Review + assign

Showing a filtered list of roles because your permissions include a condition. [Learn more](#)
[View my access](#)

Selected role Storage Blob Data Contributor

Assign access to
☐ User, group, or service principal
☒ Managed identity

Members
[+ Select members](#)

Name	Object ID	Type
No members selected		

Description
Optional

[Review + assign](#) [Previous](#) [Next](#)

Select managed identities

Some results might be hidden due to your ABAC condition.

Subscription * LabsKraft

Managed identity
Access Connector for Azure Databricks (1)

[Search by name](#)

Selected members:
 sagarconnector
/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/... [Remove](#)

[Select](#) [Close](#) [Feedback](#)

Creating sagar_cap2 using external location and Access connector:

Microsoft Azure databricks

Search data, notebooks, recent, and more... CTRL + P tred_d4

New

- Workspace
- Recents
- Catalog
- Jobs & Pipelines
- Compute
- Marketplace

SQL

- SQL Editor
- Queries
- Dashboards
- Genie
- Alerts
- Query History
- SQL Warehouses

Data Engineering

- Job Runs
- Data Ingestion

AI/ML

- Playground
- Experiments
- Features

An external location allows you to access your data stored in cloud storage (e.g. Azure Data Lake Storage). You will need the cloud storage path and a paired credential (e.g. managed identity) which gives access to that path [Learn more](#)

External location name*
sagar_location2

Storage type*
Azure Data Lake Storage

URL*
Enter the bucket path that you want to use as the external location. Note: This must be an ADLS Gen2 storage account with a hierarchical namespace
abfss://cat@sagarcap2.dfs.core.windows.net [Copy from DBFS](#)

Storage credential* [Learn more](#)
Provide a storage credential capable of accessing the URL
[+ Create new storage credential](#)

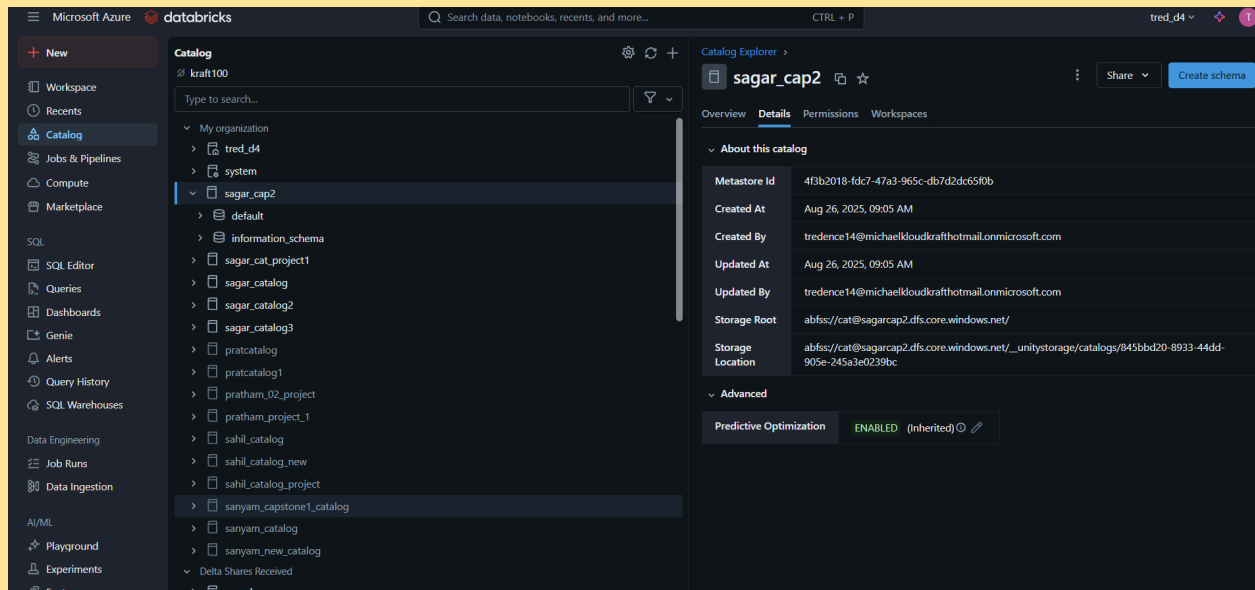
Access connector ID [Learn more](#)
/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/LabsKraft2027/providers/Microsoft.Databricks/accessConnectors/sagarconnector

User assigned managed identity ID (optional)

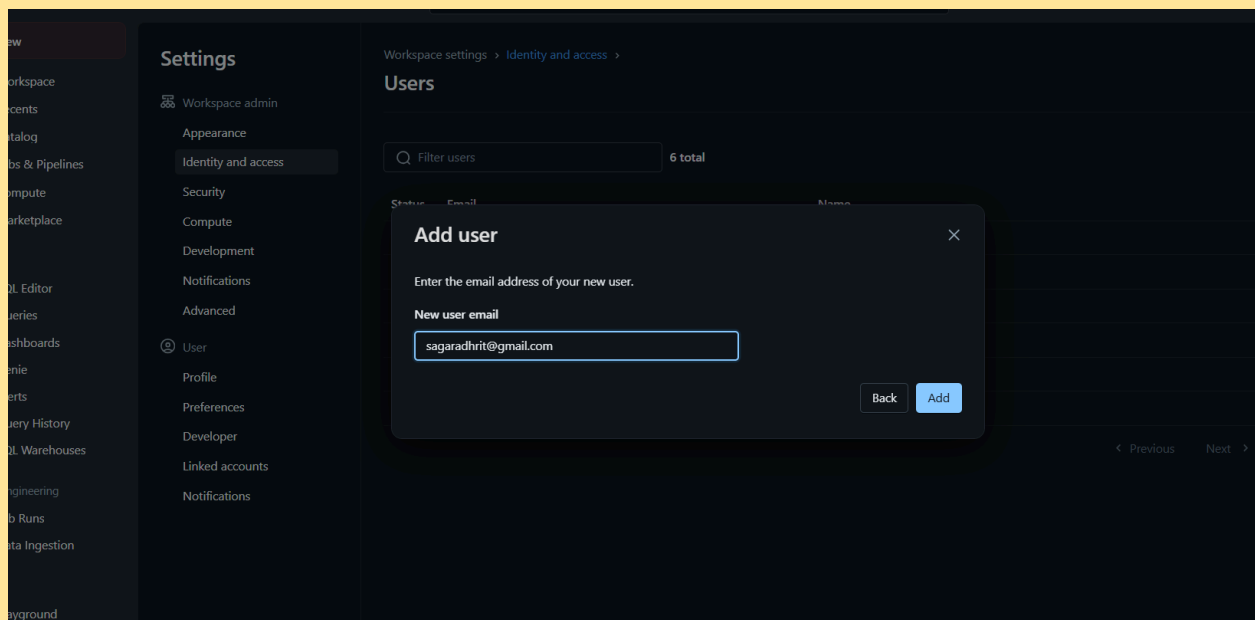
Comment

[Advanced Options](#)

Creation of catalog sagar_cap2:



Adding user and making groups:



Settings

Workspace admin

Appearance

Identity and access

Security

Compute

Development

Notifications

Advanced

User

Profile

Preferences

Developer

Linked accounts

Notifications

Workspace settings > Identity and access >

Groups

Filter groups

5 total

Add group

Name	Members	Source
admins	7	System ⓘ
analysts_90	0	Account ⓘ
auditors_90	0	Account ⓘ
data_engineer_group90	0	Account ⓘ
users	16	System ⓘ

< Previous

Next >

20 / page

After creation of dlt table:

Just now (2s)

1

SQL

```
%sql
-- Data Engineers: Full control on Bronze & Silver tables
GRANT ALL PRIVILEGES ON TABLE sagar_cap2.sales_analytics.bronze_products TO data_engineer_group90;
GRANT ALL PRIVILEGES ON TABLE sagar_cap2.sales_analytics.silver_products TO data_engineer_group90;

-- Analysts: Read-only on Gold table
GRANT SELECT ON TABLE sagar_cap2.sales_analytics.gold_top_products_by_region TO analysts_90;

-- Auditors: Read-only on Gold tables
GRANT SELECT ON TABLE sagar_cap2.sales_analytics.gold_daily_revenue TO auditors_90;
GRANT SELECT ON TABLE sagar_cap2.sales_analytics.gold_monthly_sales TO auditors_90;
> See performance \(5\)
```

Hide 4 additional results

OK

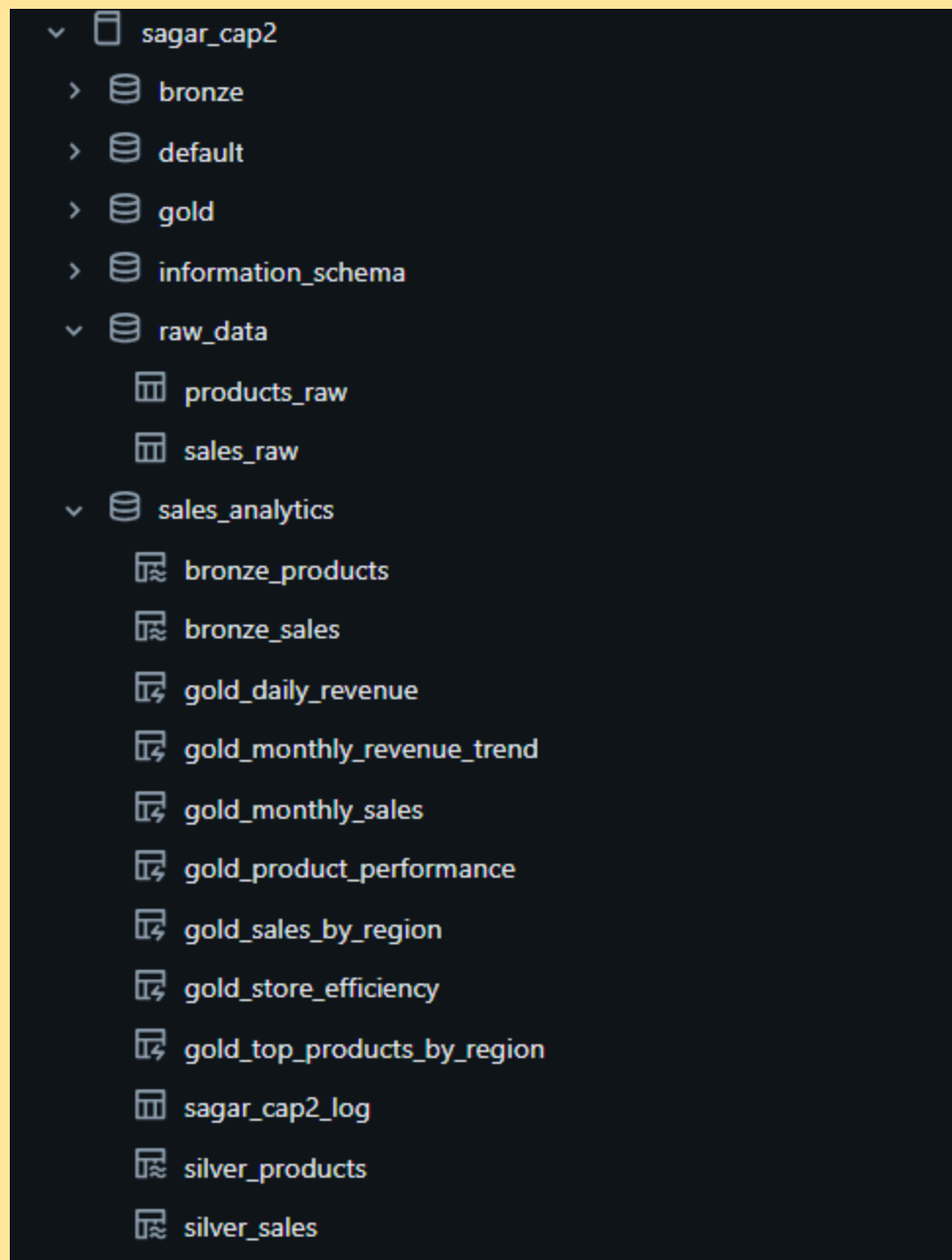
OK

OK

OK

OK

[Shift+Enter] to run and move to next cell



Migration & File Recovery

- Created **External Location** `sagar_rawbackup_project2` in Unity Catalog.
- Linked to storage **sagarstorageo** using existing **Access Connector**.


```
# Set up configs for Destination Storage
spark.conf.set(
    "fs.azure.account.key.sagarstorage0.dfs.core.windows.net",
    "YsOugmaRgeTlFbtLDEIiq60NLcAu9SuTh0W+R0tddpCuJTSzntvjSg69q6TpNyJz/eniSd399G7F+ASTJBVm0g=="
)

# Define paths
source_path = "abfss://raw@sagarstorage0.dfs.core.windows.net/sales/"
dest_path   = "abfss://rawbackup@sagarstorage0.dfs.core.windows.net/sales/"

# Read data from source (all files)
df = spark.read.format("csv").load(source_path)

# Write data into destination
(
    df.write
    .format("csv")
    .mode("overwrite")
    .save(dest_path)
)

print("✅ Migration of all files completed successfully!")
```

▶ (3) Spark Jobs

▶ df: pyspark.sql.connect.dataframe.DataFrame = [_c0: string, _c1: string ... 5 more fields]

✅ Migration of all files completed successfully!

The screenshot shows the 'rawbackup' container interface. The left sidebar has 'Overview' selected. The main area shows the 'sales' folder with a table of files. The table has columns: Name, Last modified, Access tier, Blob type, Size, and Lease state. There are 6 items listed, including a folder '[.]' and several CSV files.

☐	Name	Last modified	Access tier	Blob type	Size	Lease state	
☐	[-.]						...
☐	[_SUCCESS]	8/26/2025, 2:32:39 PM	Hot (Inferred)	Block blob	0	Available	...
☐	[_committed_5105433241976177575]	8/26/2025, 2:32:39 PM	Hot (Inferred)	Block blob	285 B	Available	...
☐	[_started_5105433241976177575]	8/26/2025, 2:32:38 PM	Hot (Inferred)	Block blob	0	Available	...
☐	part-00000-tid-5105433241976177575-76f28236-71cf-41f5-a329-028a...	8/26/2025, 2:32:39 PM	Hot (Inferred)	Block blob	99.09 KiB	Available	...
☐	part-00001-tid-5105433241976177575-76f28236-71cf-41f5-a329-028a...	8/26/2025, 2:32:39 PM	Hot (Inferred)	Block blob	99.03 KiB	Available	...
☐	part-00002-tid-5105433241976177575-76f28236-71cf-41f5-a329-028a...	8/26/2025, 2:32:39 PM	Hot (Inferred)	Block blob	49.48 KiB	Available	...

You can see we have received all the files in rawbackup container.

Creating External Table in Databricks(Quick summary)

In **Azure Databricks**, I set up an **external location** for my storage container so that data stored in **ADLS Gen2 (Azure Data Lake Storage)** can be directly queried as external tables.

1. Storage Setup

- I have a storage account named **sagarstorage0**.
- Inside it, there is a container called **rawbackup**, which contains two folders:

- **sales/** → stores sales data in CSV format.
- **products/** → stores product data in JSON format.

2. Create Storage Credential & External Location

- First, I created a **storage credential** using the **storage account key**.
- Then, I defined an **external location** pointing to the rawbackup container.
- This allows Databricks to securely access all data stored under that container.

3. Create External Tables

- After linking the container, I created external tables for both datasets:
 - **sales_raw** → Reads data from rawbackup/sales/ as **CSV**.
 - **products_raw** → Reads data from rawbackup/products/ as **JSON** (with schema inference and multiline enabled).

4. Outcome

- Now, whenever new files are dropped into the sales/ or products/ folders, they will be automatically available in the corresponding external tables (sales_raw and products_raw).
- This setup helps maintain a **centralized, scalable data lake** where Databricks queries data directly from ADLS without needing to copy or duplicate files.



✓ Just now (29s)

4

SQL



```
%sql
-- Drop if table already exists
DROP TABLE IF EXISTS sagar_cap2.raw_data.sales_raw;

-- Create Sales Raw Table (CSV files)
CREATE TABLE sagar_cap2.raw_data.sales_raw
USING CSV
OPTIONS (
  header = "true",
  inferSchema = "true"
)
LOCATION 'abfss://rawbackup@sagarstorage0.dfs.core.windows.net/sales/';

-- Drop if table already exists
DROP TABLE IF EXISTS sagar_cap2.raw_data.products_raw;

-- Create Products Raw Table (JSON files)
CREATE TABLE sagar_cap2.raw_data.products_raw
USING JSON
OPTIONS (
  multiline = "true"
)
LOCATION 'abfss://rawbackup@sagarstorage0.dfs.core.windows.net/products/';
```

▶ (3) Spark Jobs

11:36 AM (1s) 3 Python

```
display(spark.table('sagar_cap2.raw_data.sales_raw'))
```

(1) Spark Jobs

Table +

	sale_id	product_id	region	store_id	quantity	sales_amount	sale_date
1	1	157	APAC	Store_31	16	1289.03	2025-08-11 21:21
2	2	65	APAC	Store_9	8	1301.48	2025-08-12 02:41
3	3	416	LATAM	Store_25	19	3047.56	2025-08-10 00:00
4	4	501	US	Store_29	18	1955.74	2025-07-28 09:41
5	5	244	MEA	Store_6	20	2362.98	2025-08-15 00:21
6	6	240	EU	Store_22	8	1516.96	2025-08-25 05:51
7	7	854	EU	Store_24	10	178.92	2025-07-28 09:51
8	8	69	US	Store_9	7	246.4	2025-08-10 22:41
9	9	654	EU	Store_3	14	138.23	2025-08-02 05:51
10	10	277	US	Store_8	15	145.13	2025-08-15 19:01
11	11	584	LATAM	Store_4	13	2516.03	2025-08-08 13:21
12	12	432	APAC	Store_1	13	241.39	2025-07-27 22:21
13	13	797	US	Store_25	11	1914.19	2025-08-05 00:51
14	14	64	MEA	Store_32	15	2545.81	2025-08-09 06:41
15							

11:31 AM (1s) 8 Python

```
display(spark.table('sagar_cap2.raw_data.products_raw'))
```

(1) Spark Jobs

Table +

	category	discount_rate	price	product_id	product_name
1	Furniture	0.21	182.68	1	Product_5SQ8
2	Toys	0.21	115.46	2	Product_DX56
3	Furniture	0.09	121.25	3	Product_X72V
4	Electronics	0.1	31.42	4	Product_4Q7G
5	Furniture	0.03	190.8	5	Product_SDGU
6	Clothing	0.04	50.1	6	Product_LGGB
7	Toys	0.01	83.02	7	Product_30HG
8	Books	0.29	59.37	8	Product_O1MD
9	Electronics	0.01	36.16	9	Product_NF1Q
10	Toys	0.13	162.73	10	Product_KCI5
11	Grocery	0.15	121.37	11	Product_98SC
12	Electronics	0.26	185.62	12	Product_Y15D
13	Books	0.04	61.01	13	Product_SAJQ
14	Clothing	0.06	92.49	14	Product_QJTP
15	Grocery	0.26	31.1	15	Product_1BGF

1,000 rows | 1.08s runtime

Refreshed 12 min

◆ Bronze Layer (Raw Ingestion)

- Purpose: Capture **raw files** from ADLS using **Auto Loader**.
- Tables:
 - **bronze_sales** → Streams raw CSV sales files.
 - **bronze_products** → Streams raw JSON product files.
- Features:
 - Continuous ingestion (readStream)
 - Schema evolution enabled (schemaEvolutionMode="addNewColumns")

```
import dlt
from pyspark.sql import functions as F
from pyspark.sql.types import *

storage_account_name='raw@sagarstorageo'
storage_account_key='YsOugmaRgeTlFbtlDEliq6oNLCau9SuThoW+Rotdd
pCuJTSzntvjSg69q6TpNyJz/eniSd399G7F+AStJBVmog=='
spark.conf.set(
    f'fs.azure.account.key.{storage_account_name}.dfs.core.windows.net',
    storage_account_key
)

@dlt.table(
    name="bronze_sales",
```

```

    comment="Raw sales data ingested continuously from ADLS"
)
def bronze_sales():
    return (
        spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .option("cloudFiles.inferColumnTypes", "true")
        .option("cloudFiles.schemaLocation",
"abfss://raw@sagarstorageo.dfs.core.windows.net/_schemas/sales")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
        .load("abfss://raw@sagarstorageo.dfs.core.windows.net/sales/")
    )

@dlt.table(
    name="bronze_products",
    comment="Raw products data ingested continuously from ADLS"
)
def bronze_products():
    return (
        spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("multiline", "true")
        .option("cloudFiles.inferColumnTypes", "true")
        .option("cloudFiles.schemaLocation",
"abfss://raw@sagarstorageo.dfs.core.windows.net/_schemas/products")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
        .load("abfss://raw@sagarstorageo.dfs.core.windows.net/products/")
    )

```

◆ Silver Layer (Clean & Standardized)

- Purpose: Apply **data cleaning, type casting, deduplication, and validations**.
- Tables:
 - **silver_sales** →

- Casts datatypes (sale_id, sales_amount, sale_date, etc.)
- Extracts store_id number
- Enriches with derived fields (sale_month_year, sale_year, sale_month_name)
- Deduplicates on sale_id
- Data quality checks (sales_amount >= 0, quantity > 0)
- **silver_products** →
 - Cleans product data
 - Casts datatypes (price, discount_rate)
 - Normalizes product names
 - Deduplicates on product_i

```

@dlt.table(

    name="silver_sales",

    comment="Cleaned, validated, and enriched sales data"

)

@dlt.expect_or_drop("valid_sale_id", "sale_id IS NOT NULL")

@dlt.expect_or_drop("valid_sale_date", "sale_date IS NOT NULL")

@dlt.expect("valid_sales_amount", "sales_amount >= 0")

@dlt.expect("valid_quantity", "quantity > 0")

def silver_sales():

    df = dlt.read_stream("bronze_sales")

    df_transformed = df \

        .withColumn('sale_id', F.col('sale_id').cast('int')) \

        .withColumn('product_id', F.col('product_id').cast('int')) \

        .withColumn('region', F.col('region').cast('string')) \

        .withColumn('quantity', F.col('quantity').cast('int')) \

        .withColumn('sales_amount', F.col('sales_amount').cast('double')) \

        .withColumn('sale_date', F.col('sale_date').cast('timestamp')) \

        .withColumn("store_id", F.regexp_extract(F.col("store_id"),

r"Store_(\d+)", 1).cast("int"))

```

```

df_enriched = df_transformed \

    .withColumn("sale_month_year", (F.year(F.col("sale_date")) * 100 +
F.month(F.col("sale_date"))).cast("int")) \

    .withColumn("sale_month", F.month(F.col("sale_date")).cast("int")) \

    .withColumn("sale_month_name", F.date_format(F.col("sale_date"),
"MMMM")) \

    .withColumn("sale_year", F.year(F.col("sale_date")).cast("int"))

return df_enriched.withWatermark("sale_date", "10
hours").dropDuplicates(["sale_id"])

```

```

@dlt.table(

    name="silver_products",

    comment="Cleaned and validated products data"

)

@dlt.expect_or_drop("valid_product_id", "product_id IS NOT NULL")

def silver_products():

    df = dlt.read_stream("bronze_products")

    df_transformed = df \

        .withColumn('category', F.col('category').cast('string')) \

        .withColumn('discount_rate', F.col('discount_rate').cast('double')) \

        .withColumn('price', F.col('price').cast('double')) \

```



```
.withColumn('product_id', F.col('product_id').cast('int')) \

.withColumn('product_name', F.substring(F.col("product_name"), 9,
100))

return df_transformed.dropDuplicates(["product_id"])
```

Gold Layer (Business Aggregates)

- Purpose: Deliver **business-ready KPIs and insights** for analytics/reporting.
- Tables:
 1. **gold_monthly_sales** → Monthly sales totals by product & category.
 2. **gold_sales_by_region** → Regional sales & count of transactions.
 3. **gold_daily_revenue** → Daily revenue & quantity per region & store.
 4. **gold_monthly_revenue_trend** → Revenue trend by month & region.
 5. **gold_product_performance** → Product performance + discount insights.
 6. **gold_top_products_by_region** → Top 5 products per region.
 7. **gold_store_efficiency** → Store-level KPIs (avg sales per transaction, avg discount).

```
@dlt.table(
    name="gold_monthly_sales",
    comment="Monthly aggregated sales data per product"
)
def gold_monthly_sales():
    sales = dlt.read("silver_sales")
    products = dlt.read("silver_products")
```

```

    return (
        sales.join(products, "product_id", "inner")
        .groupBy("sale_year", "sale_month", "sale_month_name",
"product_name", "category")
        .agg(
            F.sum("sales_amount").alias("total_sales_amount"),
            F.sum("quantity").alias("total_quantity_sold")
        )
    )
)

@dlt.table(
    name="gold_sales_by_region",
    comment="Aggregated sales data by region and product category"
)
def gold_sales_by_region():
    sales = dlt.read("silver_sales")
    products = dlt.read("silver_products")
    return (
        sales.join(products, "product_id", "inner")
        .groupBy("region", "category")
        .agg(
            F.sum("sales_amount").alias("total_sales_amount"),
            F.count("sale_id").alias("number_of_sales")
        )
    )
)

@dlt.table(
    name="gold_daily_revenue",
    comment="Daily revenue and quantity aggregated per region and
store"
)
def gold_daily_revenue():
    sales = dlt.read("silver_sales")
    return (
        sales.groupBy(F.to_date(F.col("sale_date")).alias("sale_day"),
"region", "store_id")
        .agg(
            F.sum("sales_amount").alias("daily_revenue"),

```

```

        F.sum("quantity").alias("total_quantity")
    )
)

@dlt.table(
    name="gold_monthly_revenue_trend",
    comment="Monthly revenue aggregated per region"
)
def gold_monthly_revenue_trend():
    sales = dlt.read("silver_sales")
    return (
        sales.groupBy("sale_month_year", "region")
        .agg(F.sum("sales_amount").alias("monthly_revenue"))
    )

@dlt.table(
    name="gold_product_performance",
    comment="Product-level sales performance with discount insights"
)
def gold_product_performance():
    sales = dlt.read("silver_sales")
    products = dlt.read("silver_products")
    return (
        sales.join(products, "product_id", "inner")
        .groupBy("product_id", "product_name", "category")
        .agg(
            F.sum("sales_amount").alias("total_sales"),
            F.sum("quantity").alias("total_quantity"),
            F.avg("discount_rate").alias("avg_discount_rate")
        )
    )

@dlt.table(
    name="gold_top_products_by_region",
    comment="Top 5 products by total sales per region"
)
def gold_top_products_by_region():
    from pyspark.sql.window import Window

```

```

sales = dlt.read("silver_sales")
products = dlt.read("silver_products")
ranked = (
    sales.join(products, "product_id", "inner")
        .groupBy("region", "product_name")
        .agg(F.sum("sales_amount").alias("total_sales"))
)
windowSpec =
Window.partitionBy("region").orderBy(F.desc("total_sales"))
return (
    ranked.withColumn("rank",
F.row_number().over(windowSpec))
        .filter(F.col("rank") <= 5)
)

@dlt.table(
    name="gold_store_efficiency",
    comment="Store-level KPIs for benchmarking"
)
def gold_store_efficiency():
    sales = dlt.read("silver_sales")
    products = dlt.read("silver_products")
    return (
        sales.join(products, "product_id", "inner")
            .groupBy("store_id", "region")
            .agg(
                (F.sum("sales_amount") /
F.countDistinct("sale_id")).alias("avg_sales_per_transaction"),
                F.avg("discount_rate").alias("avg_discount_applied")
            )
    )

```

Pipeline Settings in Triggered mode:

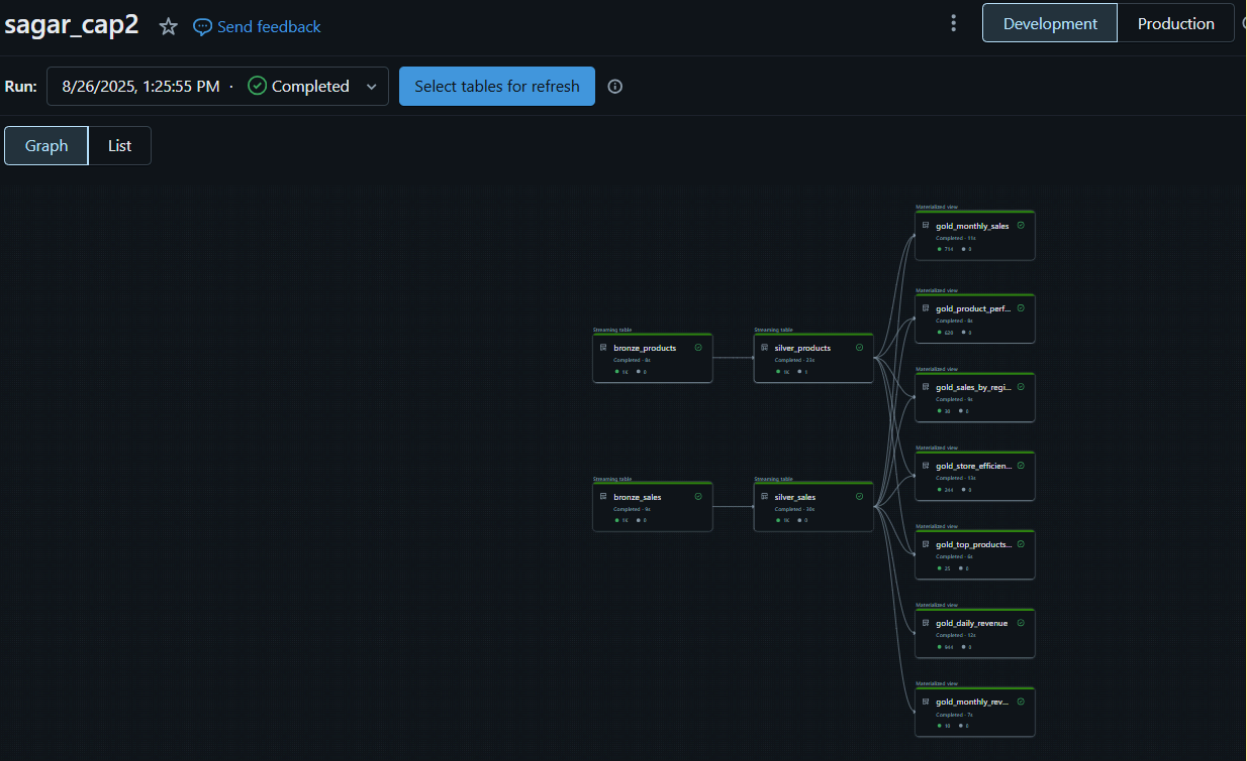
Pipeline settings [Send feedback](#)

```
1 {
2   "id": "eb1f4e94-370d-44c9-97e7-bd866a6ae4e5",
3   "pipeline_type": "WORKSPACE",
4   "development": true,
5   "event_log": {
6     "catalog": "sagar_cap2",
7     "schema": "sales_analytics",
8     "name": "sagar_cap2_log"
9   },
10  "continuous": false,
11  "channel": "CURRENT",
12  "photon": true,
13  "libraries": [
14    {
15      "notebook": {
16        "path": "/Users/tredence14@michaelcloudkrafthotmail.onmicrosoft.com/Capstone2/Untitled Notebook 2025-08-
26 11:52:16"
17      }
18    }
19  ],
20  "name": "sagar_cap2",
21  "serverless": true,
22  "catalog": "sagar_cap2",
23  "schema": "sales_analytics"
24 }
```

[illegible]

✓ Resolution: External Table & Location Issue

- Dropped the previous external table which was pointing to **raw@sagarstorageo**.
- Created a new container **rawbackup** in the same storage account.
- Migrated all files from old location to **rawbackup**.
- Set **rawbackup@sagarstorageo** as the new **External Location**.
- Now, external tables point to this new location, ensuring **smooth ingestion and schema evolution**.



Graph

List

bronze_products

Completed 1h

12 0 1

silver_products

Completed 1h

12 0 1

bronze_sales

Completed 1h

12 0 1

silver_sales

Completed 1h

12 0 1

gold_monthly_sales

Completed 1h

10 0 1

gold_product_perf...

Completed 1h

10 0 1

gold_sales_by_regi...

Completed 1h

10 0 1

gold_store_efficien...

Completed 1h

10 0 1

gold_top_products...

Completed 1h

10 0 1

gold_daily_revenue

Completed 1h

10 0 1

gold_monthly_rev...

Completed 1h

10 0 1

Event log

Query history

All

Info

Warning

Error

Filter...

5 hours ago

flow_progress

Flow 'sagar_cap2.sales_analytics.gold_store_efficiency' has COMPLETED.

5 hours ago

flow_progress

Flow 'sagar_cap2.sales_analytics.gold_monthly_sales' has COMPLETED.

5 hours ago

flow_progress

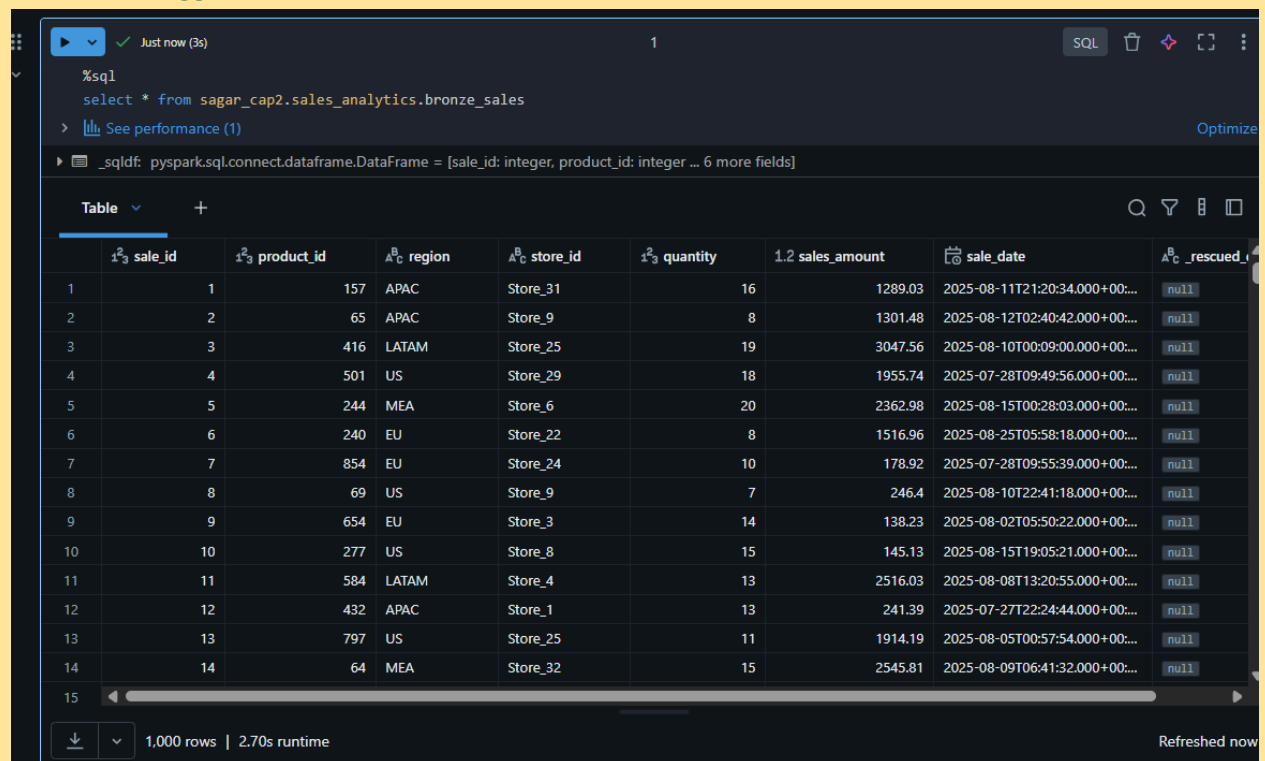
Flow 'sagar_cap2.sales_analytics.gold_monthly_revenue_trend' has COMPLETED.

5 hours ago

update_progress

Update 351ad7 is COMPLETED.

After first trigger :



Just now (3s) 1 SQL

```
%sql
select * from sagar_cap2.sales_analytics.bronze_sales
```

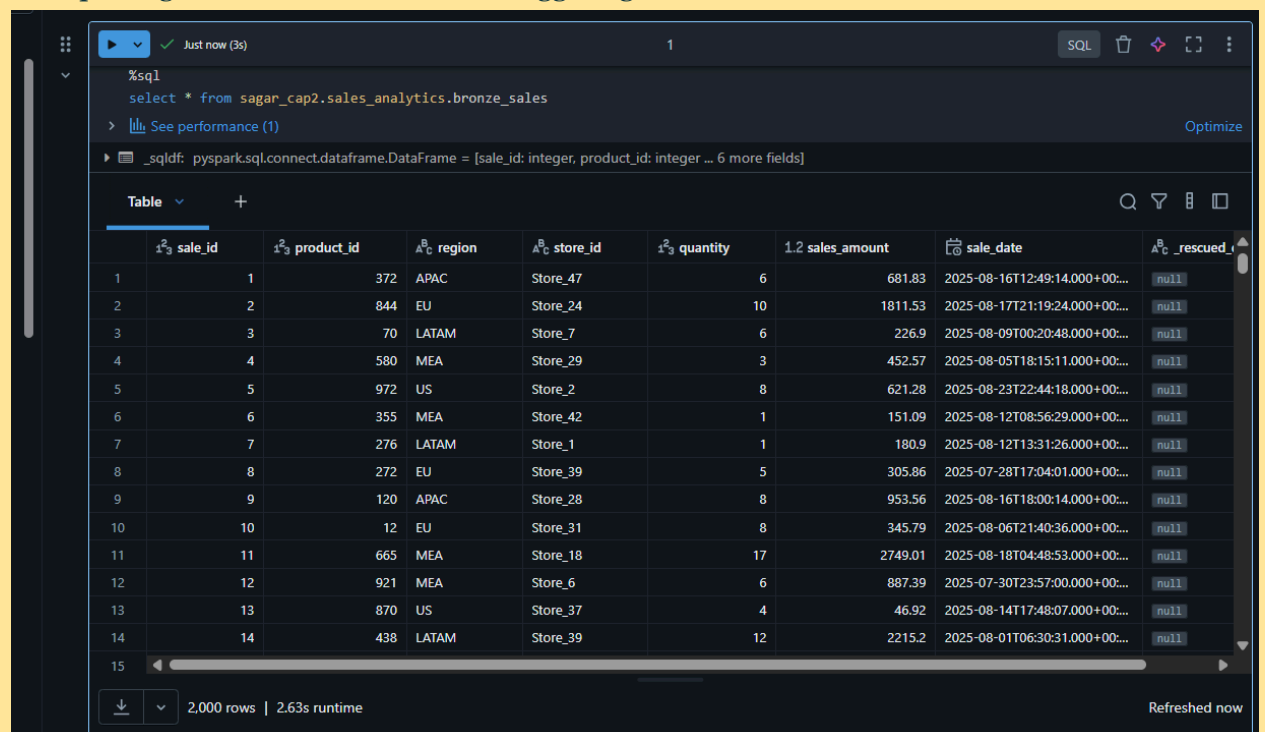
> [See performance \(1\)](#) Optimize

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [sale_id: integer, product_id: integer ... 6 more fields]

	sale_id	product_id	region	store_id	quantity	sales_amount	sale_date	_rescued
1	1	157	APAC	Store_31	16	1289.03	2025-08-11T21:20:34.000+00:...	null
2	2	65	APAC	Store_9	8	1301.48	2025-08-12T02:40:42.000+00:...	null
3	3	416	LATAM	Store_25	19	3047.56	2025-08-10T00:09:00.000+00:...	null
4	4	501	US	Store_29	18	1955.74	2025-07-28T09:49:56.000+00:...	null
5	5	244	MEA	Store_6	20	2362.98	2025-08-15T00:28:03.000+00:...	null
6	6	240	EU	Store_22	8	1516.96	2025-08-25T05:58:18.000+00:...	null
7	7	854	EU	Store_24	10	178.92	2025-07-28T09:55:39.000+00:...	null
8	8	69	US	Store_9	7	246.4	2025-08-10T22:41:18.000+00:...	null
9	9	654	EU	Store_3	14	138.23	2025-08-02T05:50:22.000+00:...	null
10	10	277	US	Store_8	15	145.13	2025-08-15T19:05:21.000+00:...	null
11	11	584	LATAM	Store_4	13	2516.03	2025-08-08T13:20:55.000+00:...	null
12	12	432	APAC	Store_1	13	241.39	2025-07-27T22:24:44.000+00:...	null
13	13	797	US	Store_25	11	1914.19	2025-08-05T00:57:54.000+00:...	null
14	14	64	MEA	Store_32	15	2545.81	2025-08-09T06:41:32.000+00:...	null

1,000 rows | 2.70s runtime Refreshed now

After putting second file into and then triggering:



Just now (3s) 1 SQL

```
%sql
select * from sagar_cap2.sales_analytics.bronze_sales
```

> [See performance \(1\)](#) Optimize

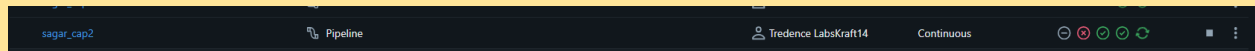
_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [sale_id: integer, product_id: integer ... 6 more fields]

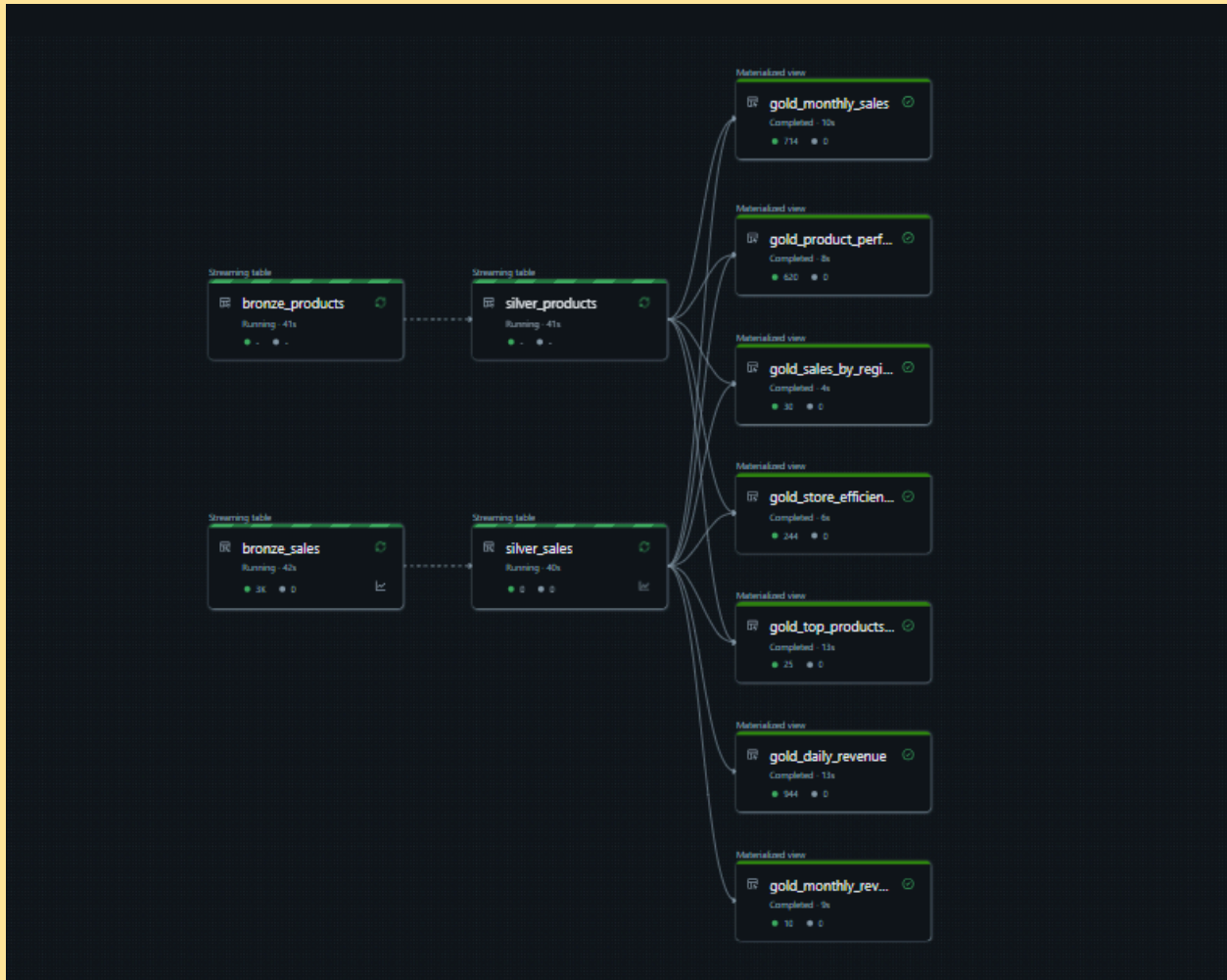
	sale_id	product_id	region	store_id	quantity	sales_amount	sale_date	_rescued
1	1	372	APAC	Store_47	6	681.83	2025-08-16T12:49:14.000+00:...	null
2	2	844	EU	Store_24	10	1811.53	2025-08-17T21:19:24.000+00:...	null
3	3	70	LATAM	Store_7	6	226.9	2025-08-09T00:20:48.000+00:...	null
4	4	580	MEA	Store_29	3	452.57	2025-08-05T18:15:11.000+00:...	null
5	5	972	US	Store_2	8	621.28	2025-08-23T22:44:18.000+00:...	null
6	6	355	MEA	Store_42	1	151.09	2025-08-12T08:56:29.000+00:...	null
7	7	276	LATAM	Store_1	1	180.9	2025-08-12T13:31:26.000+00:...	null
8	8	272	EU	Store_39	5	305.86	2025-07-28T17:04:01.000+00:...	null
9	9	120	APAC	Store_28	8	953.56	2025-08-16T18:00:14.000+00:...	null
10	10	12	EU	Store_31	8	345.79	2025-08-06T21:40:36.000+00:...	null
11	11	665	MEA	Store_18	17	2749.01	2025-08-18T04:48:53.000+00:...	null
12	12	921	MEA	Store_6	6	887.39	2025-07-30T23:57:00.000+00:...	null
13	13	870	US	Store_37	4	46.92	2025-08-14T17:48:07.000+00:...	null
14	14	438	LATAM	Store_39	12	2215.2	2025-08-01T06:30:31.000+00:...	null

2,000 rows | 2.63s runtime Refreshed now

you can see that no of rows has been increased

Now running in continuous mode after putting all the files:





It's still running and bronze table is showing 5000 rows:

Just now (3s)

1

SQL

```
%sql
select * from sagar_cap2.sales_analytics.bronze_sales
```

See performance (1)

Optimize

```
_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [sale_id: integer, product_id: integer ... 6 more fields]
```

	sale_id	product_id	region	store_id	quantity	sales_amount	sale_date	_rescued_r
1	1	737	APAC	Store_40	16	655.38	2025-07-31T01:49:56.000+00...	null
2	2	185	LATAM	Store_48	6	872.66	2025-08-17T10:44:11.000+00...	null
3	3	18	EU	Store_20	11	317.64	2025-08-19T11:43:43.000+00...	null
4	4	744	EU	Store_21	2	276.58	2025-08-19T19:01:32.000+00...	null
5	5	878	US	Store_44	4	250.87	2025-08-14T19:25:43.000+00...	null
6	6	436	MEA	Store_13	2	139.47	2025-08-07T17:03:20.000+00...	null
7	7	69	APAC	Store_48	10	384.03	2025-08-12T16:09:04.000+00...	null
8	8	160	APAC	Store_20	4	477.14	2025-08-08T05:08:51.000+00...	null
9	9	62	EU	Store_30	1	25.4	2025-08-20T15:21:51.000+00...	null
10	10	942	EU	Store_22	18	2252.79	2025-07-31T02:22:24.000+00...	null
11	11	189	MEA	Store_42	18	2905.3	2025-08-03T22:43:27.000+00...	null
12	12	491	MEA	Store_35	18	3106.7	2025-08-08T11:39:16.000+00...	null
13	13	193	MEA	Store_21	17	3313.8	2025-08-08T21:27:17.000+00...	null
14	14	177	MEA	Store_29	12	107.04	2025-08-21T00:03:05.000+00...	null
15								

5,000 rows | 2.81s runtime

Refreshed now

Monitoring & Troubleshooting

event log:

Graph

List

Event log

Query history

All

Info

Warning

Error

Filter...

5 hours ago	flow_progress	Flow 'sagar_cap2.sales_analytics.gold_store_efficiency' has COMPLETED.
5 hours ago	flow_progress	Flow 'sagar_cap2.sales_analytics.gold_monthly_sales' has COMPLETED.
5 hours ago	flow_progress	Flow 'sagar_cap2.sales_analytics.gold_monthly_revenue_trend' has COMPLETED.
5 hours ago	update_progress	Update 351ad7 is COMPLETED.

when closely observed we will find the no. of rows processed in pipeline tables in graph.

adding notification:

Notifications

* Email(s) (comma-separated)

sagaradhr@gmail.com

On update: ☒ Success ☒ Failure ☒ Fatal failure

On flow: ☒ Failure

 The "on-update-success" alert does not apply to continuous pipelines

Add notification

Time Travel

▶ 4 minutes ago (4s) 2 SQL

%sql
DESCRIBE HISTORY sagar_cap2.sales_analytics.bronze_products;
> [See performance \(1\)](#)

Table +

	version	timestamp	userid	userName	operation	operationPar
1	2	2025-08-26T15:08:40.000+00:...	148486118910247	tredence14@michaelcloudkrafthotmail.onmicrosoft.co...	STREAMING UPDATE	> {outputMode
2	1	2025-08-26T15:08:21.000+00:...	148486118910247	tredence14@michaelcloudkrafthotmail.onmicrosoft.co...	DLT SETUP	> {pipelineId:"
3	0	2025-08-26T15:08:19.000+00:...	148486118910247	tredence14@michaelcloudkrafthotmail.onmicrosoft.co...	CREATE TABLE	> {partitionBy"

3 rows | 3.98s runtime

Refreshed 4 minu

▶

✓ 2 minutes ago (3s)

3

SQL

🗑️

⚡

🔗

```
%sql
SELECT *
FROM sagar_cap2.sales_analytics.bronze_products
TIMESTAMP AS OF '2025-08-26T15:08:40.000+00:00';
```

> [See performance \(1\)](#)

▶

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [category: string, discount_rate: double ... 4 more fields]

Table ▾

+

🔍

🏠

⌵

	Ⓐ category	1.2 discount_rate	1.2 price	1.2 product_id	Ⓐ product_name	Ⓐ rescued_data
14	Books	0.04	61.01	13	Product_SAJQ	null
15	Clothing	0.06	92.49	14	Product_QJTP	null
16	Grocery	0.26	31.1	15	Product_1BGF	null
17	Books	0.2	10.68	16	Product_SJBF	null
18	Furniture	0.01	6.19	17	Product_6DW9	null
19	Clothing	0.13	87.51	18	Product_PJG5	null
20	Furniture	0.14	175	19	Product_LC0E	null
21	Books	0.3	72.15	20	Product_9OT8	null
22	Grocery	0.13	185.42	21	Product_NX AQ	null
23	Toys	0.02	150.49	22	Product_6OY6	null
24	Clothing	0.11	78.69	23	Product_15PI	null
25	Clothing	0.28	25.27	24	Product_G6EI	null

▶

✓ 08:46 PM (3s)

4

```
%sql

DESCRIBE HISTORY sagar_cap2.sales_analytics.silver_sales;

INSERT INTO sagar_cap2.sales_analytics.silver_sales
SELECT *
FROM sagar_cap2.sales_analytics.silver_sales VERSION AS OF 2
WHERE sale_id = '12345';
```

> [See performance \(1\)](#)

Table ▾

+

	1.2 num_affected_rows	1.2 num_inserted_rows
1	0	0

creation of job for updating our compliance report:

Jobs & Pipelines > sagar_cap2_job >

sagar_cap2_job run Lakeflow Jobs UI: ON [Send feedback](#)

[Cancel job run](#) [Repair run](#)

Graph Timeline List

etl
Pending · 3m 13s
...icrosoft.com/Capstone2/etl_pipeline
kraft100

compliance_report
Blocked · 0s
...ft.com/Capstone2/compliance_report

Job run details

Job ID: 284809983212543 [Copy](#)

Job run ID: 526668242044407 [Copy](#)

Launched: Manually

Started: Aug 26, 2025, 10:25 PM

Ended: -

Duration: 3m 14s

Execution time: [View](#)

Queue duration: -

Status: Running
[Cancel](#)

Lineage: [View](#)
No lineage information for this job.
[Learn more](#)

Performance optimization: [View](#)
Disabled

[View run events](#)

Compute
kraft100

Job details

Job ID: 284809983212543 [Copy](#)

Creator: [Tredence LabsKraft14](#)

Run as: [Tredence LabsKraft14](#) [Edit](#)

Description: [Add description](#)

Lineage: [View](#)
No lineage information for this job.
[Learn more](#)

Performance optimized: ☐

Schedules & Triggers

Every day, next run at Aug 26, 2025, 11:17 PM

[Edit trigger](#) [Pause](#) [Delete](#)

Compliance Report

This notebook generates a compliance report from the sales data. It evaluates each record in the `gold_monthly_sales` table and flags those where the total sales amount exceeds 10,000. The resulting report is saved in two formats:

1. Delta Table – stored within the catalog for easy querying and further analytics.
2. CSV File – exported to the volume for external tools such as Power BI or Excel.

Finally, the notebook reads the CSV file to display the count of flagged records, providing a quick summary of compliance checks.

▶ ✓ 3 minutes ago (11s)

```
%sql
use catalog sagar_cap2;
use schema sales_analytics;
create volume if not exists volumess;
```

Python

1 minute ago (1m) 2

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark.sql(f"""
CREATE DATABASE IF NOT EXISTS sagar_cap2.sales_analytics
COMMENT 'Database to store analytics and reports'
LOCATION '/Volumes/sagar_cap2/sales_analytics/volumess/'
""")

gold_sales = spark.table("sagar_cap2.sales_analytics.gold_monthly_sales")

compliance_report = gold_sales.withColumn(
    "flagged",
    (col("total_sales_amount") > 10000).cast("int")
)

compliance_report.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("sagar_cap2.sales_analytics.compliance_report_delta")

csv_path = "/Volumes/sagar_cap2/sales_analytics/volumess/compliance_report_csv/"

compliance_report.write.option("header", True) \
    .mode("overwrite") \
    .csv(csv_path)

print("✅ Compliance report created as Delta table and CSV successfully!")
```

▶ (4) Spark Jobs

▶ ✓ Just now (38s)

3

```
import pandas as pd
df=pd.read_csv('/Volumes/sagar_recover_catalog/sales_analytics/volumess/compliance_report_csv/
part-00000-tid-353165469647112325-34bb6c3c-ed05-40d9-9ae2-fa7ac8f1ba3e-9-1-c000.csv')
display(df['flagged'].value_counts())
```

▶ df: pandas.core.frame.DataFrame = [sale_year: int64, sale_month: int64 ... 6 more fields]

0 720

Name: flagged, dtype: int64



Power BI Visualization



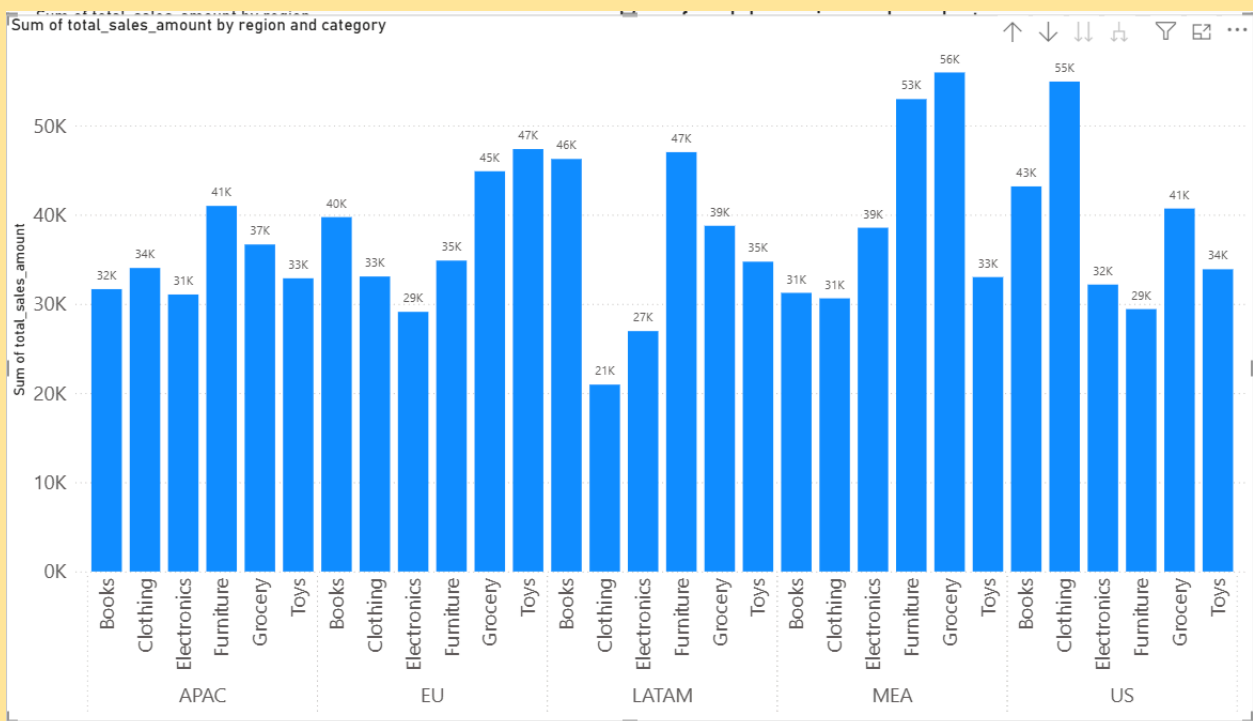
Loading of Tables:

-  Connect Power BI to the **Gold Layer** tables stored in Databricks (using the Databricks connector or JDBC/ODBC).
-  Import the following curated datasets from the Gold layer:
 -  **Daily/Monthly Sales per Region**
 -  **Device Anomaly Trend per Store**
 -  **Conversion Rate vs Device Failure**
 -  **Store Performance Tiering**

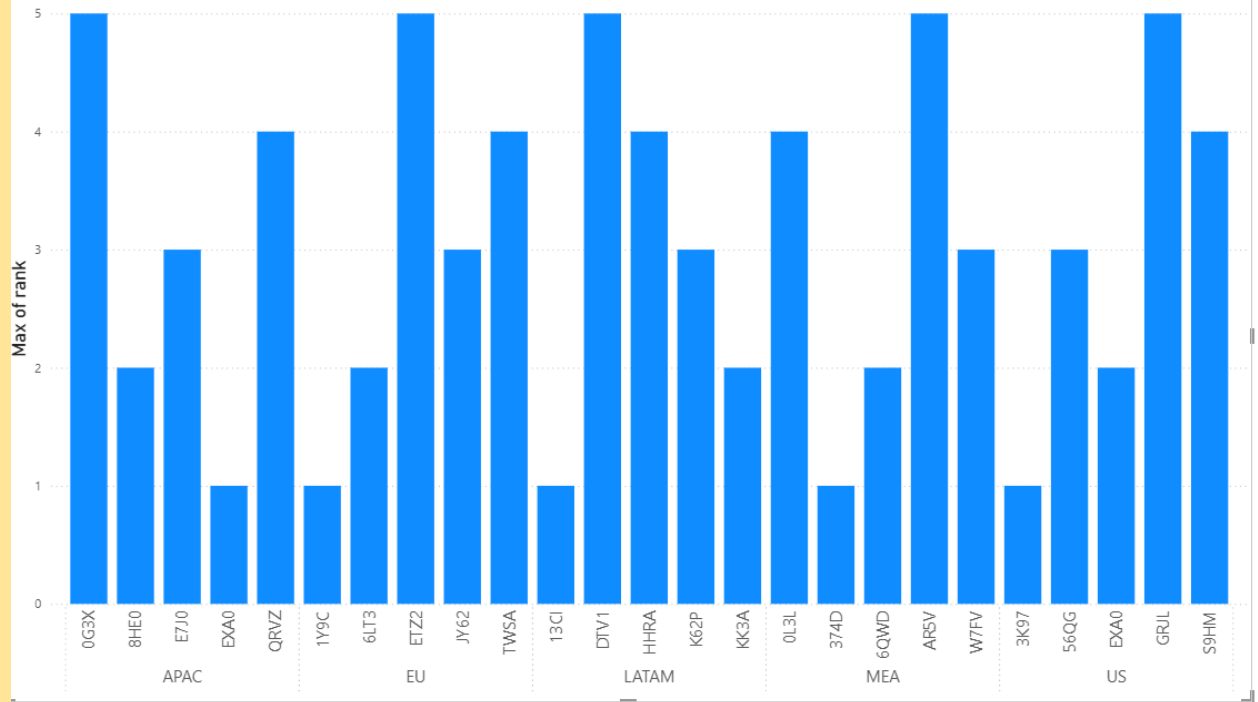
Load

- gold_daily_revenue
Waiting for {"host":"adb-3325880049345578.18.azuredatabricks.net","httpPath":"/Vsql/V1.0/Vware
- gold_monthly_revenue_trend
Waiting for {"host":"adb-3325880049345578.18.azuredatabricks.net","httpPath":"/Vsql/V1.0/Vware
- gold_monthly_sales
Waiting for {"host":"adb-3325880049345578.18.azuredatabricks.net","httpPath":"/Vsql/V1.0/Vware
- gold product performance

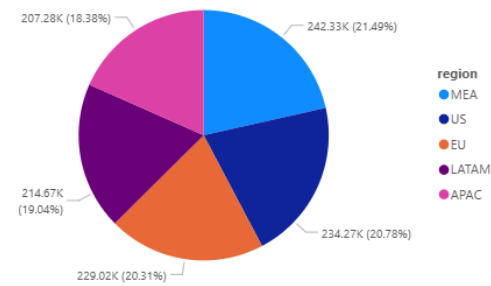
Cancel



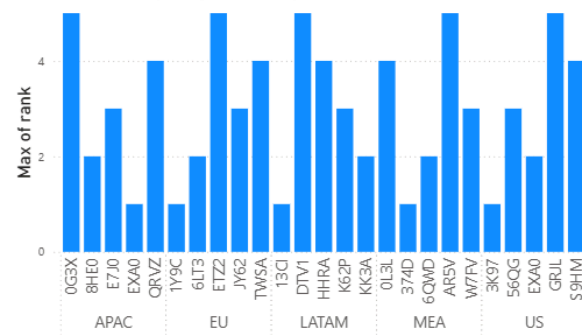
Max of rank by region and product_name



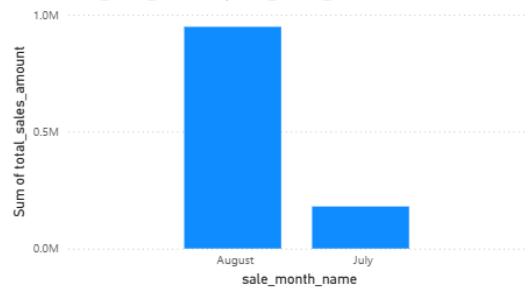
Sum of total_sales_amount by region



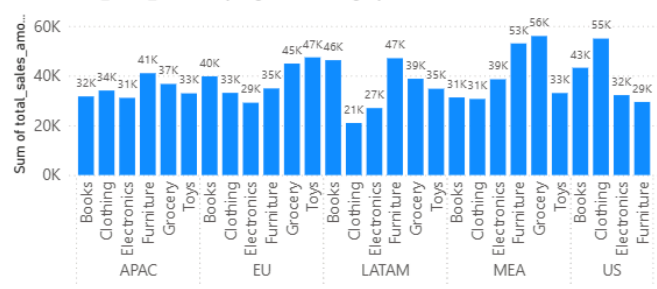
Max of rank by region and product_name

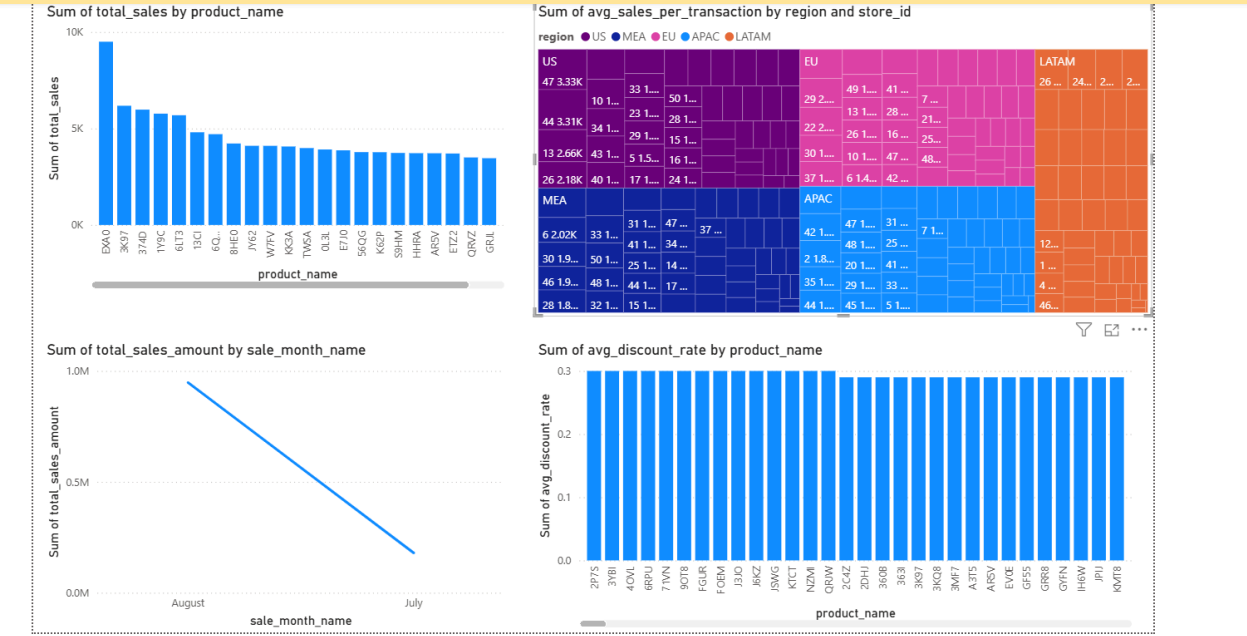
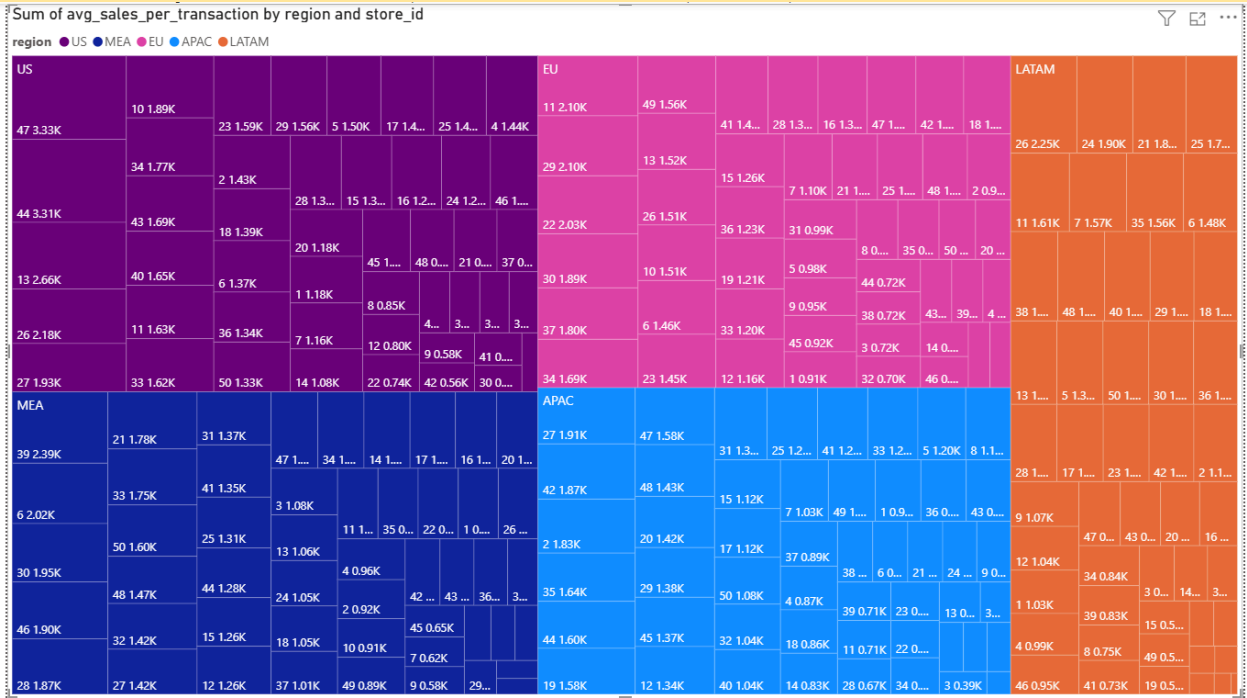


Sum of total_sales_amount by sale_month_name



Sum of total_sales_amount by region and category





created new column for gold_daily_revenue:

